



**Hochschule
Bonn-Rhein-Sieg**

*University
of Applied Sciences*

Fachbereich Informatik
Department of Computer Sciences

Abschlussarbeit

Studiengang Bachelor Informatik

Entwurf und Evaluation eines fehlertoleranten Job-Processing in Java

von

Max Urfei

Erstprüfer: Prof. Dr. Andreas Hackelöer

Zweitprüfer: M. Sc. Moritz Balg

eingereicht am TT.MM.YYYY

Inhaltsverzeichnis

Abkürzungsverzeichnis	iii
Abbildungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung	2
1.4 Hypothesen	3
1.5 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Hintergrundverarbeitung in verteilten Systemen	4
2.1.1 Synchrone vs. Asynchrone Verarbeitung	4
2.1.2 Background Job-Processing	4
2.2 Konsistenz und konkurrierende Verarbeitung	4
2.2.1 ACID	4
2.2.2 Locking	4
2.2.3 Transaktionsgrenzen	4
2.3 Fehlertoleranz und Wiederherstellung	4
2.3.1 Retry-/Backoff-Strategien	4
2.3.2 Fehlerklassifikationen	4
2.3.3 Recovery	4
2.3.4 Idempotenz	4
2.4 Zustandsmodellierung mittels State Machines	4
2.5 Stand der Technik	4
3 Methodik	5
3.1 Literaturrecherche	5
3.2 Ergebniserzeugung	5
4 Anforderungen	6
4.1 Funktionale Anforderungen	6
4.2 Qualitätsanforderungen	6
4.3 Zusicherungen	6
5 Umsetzung	7
5.1 Architektur und Design	7
5.2 Implementierung	7
5.2.1 Technologie-Stack	7
5.2.2 Zentrale Code-Entscheidungen	7

6	Evaluation	8
6.1	Fehlerszenarien	8
6.2	Durchführung	8
6.3	Ergebnisse	8
7	Diskussion	9
7.1	Gewonnene Erkenntnisse	9
7.2	Einschränkungen und Grenzen der Arbeit	9
8	Fazit und Ausblick	10
	Literaturverzeichnis	11

Abkürzungsverzeichnis

CRUD Create, Read, Update, Delete. 1

Abbildungsverzeichnis

1 Einleitung

1.1 Motivation

In modernen Softwaresystemen stellt die Verarbeitung von Hintergrundjobs einen zentralen Bestandteil vieler Backend-Architekturen dar (Kleppmann 2017, Kap. 10). Zahlreiche Geschäftsprozesse, darunter die Generierung von Reports, der Import großer Datenmengen, periodische Abrechnungsläufe oder das Versenden asynchroner Benachrichtigungen, erfordern eine Verarbeitung, die entkoppelt vom synchronen Request-Response-Zyklus stattfindet. Insbesondere in industrienahen und sicherheitskritischen Anwendungen, etwa im Bereich der luftfahrttechnischen Wartungsdokumentation, müssen solche Prozesse nicht nur funktional korrekt ablaufen, sondern darüber hinaus zuverlässig, nachvollziehbar und fehlertolerant sein.

1.2 Problemstellung

In der Praxis können bei der Verarbeitung von Hintergrundjobs jederzeit Teilausfälle auftreten. Prozesse können während der Verarbeitung unerwartet terminieren, Datenbankverbindungen können kurzzeitig unterbrochen werden oder identische Anfragen werden zum Beispiel infolge von clientseitigen Wiederholungsversuchen mehrfach an das System übermittelt oder Ergebnisse bzw. Antworten gehen verloren (Kleppmann 2017, Kap. 8). Ohne geeignete Mechanismen führen derartige Szenarien zu inkonsistenten Systemzuständen. Beispielsweise verbleiben Jobs im Status „laufend“, obwohl kein Worker sie mehr aktiv verarbeitet, Ergebnisse werden doppelt erzeugt oder Aufträge gehen vollständig verloren.

Bestehende Frameworks und Bibliotheken, etwa Quartz Scheduler (Terracotta Inc. (Hrsg.) 2026) oder Spring Batch (Broadcom Inc. (Hrsg.) 2026), adressieren Teilaspekte der Hintergrundverarbeitung, setzen jedoch voraus, dass grundlegende Architekturentscheidungen bewusst und korrekt getroffen werden. So muss unter anderem entschieden werden, wie sichergestellt wird, dass ein Job nach einem Prozessabsturz nicht verloren geht, wie verhindert wird, dass zwei Worker denselben Job gleichzeitig verarbeiten, wie das System doppelte Aufträge erkennt und redundante Ergebnisse vermeidet und wie es sich bei kurzzeitigen Datenbankausfällen verhält. Keine der genannten Lösungen beantwortet diese Fragen vollständig. Sie erfordern vielmehr ein durchdachtes Zusammenspiel aus einem persistenten Statusmodell, klar definierten Transaktionsgrenzen, geeigneten Locking-Strategien, Mechanismen zur Vermeidung doppelter Ausführung sowie Retry- und Backoff-Policies. Die zentrale Herausforderung liegt darin, diese Mechanismen so zu entwerfen und zu kombinieren, dass das System definierte Zusicherungen unter allen relevanten Fehlerszenarien einhält.

Die beschriebene Problematik ist von hoher praktischer und industrieller Relevanz. Nahezu jedes Backend-System, das über einfache Create, Read, Update, Delete (CRUD)-Operationen hinausgeht, muss eine Form der zuverlässigen Hintergrundverarbeitung mit-

tels Transaktionen implementieren (Richardson 2018, Kap. 4). Fehler in diesem Bereich führen zu Datenverlust, inkonsistenten Geschäftszuständen sowie unnötigem manuellem Eingriff und können insbesondere in regulierten Domänen wie der Luftfahrt Compliance-Verstöße nach sich ziehen. Auch im Kontext von Cloud-nativen Architekturen ist dieses Thema relevant. Horizontale Skalierung, Container-Neustarts und kurzzeitige Netzwerkunterbrechungen können auftreten und müssen behandelt werden. Daher bilden die in dieser Arbeit behandelten Konzepte auch die Grundlage für den zuverlässigen Betrieb von Backend-Systemen in Cloud-Umgebungen. Trotz dieser Relevanz existiert wenig praxisorientierte und zugleich evaluierte Literatur, die den gesamten Entwurfsprozess eines solchen Systems, von der Anforderungsdefinition über die Architekturentscheidungen bis hin zum systematischen Fehlervalidierung, nachvollziehbar dokumentiert und empirisch überprüft.

Daraus ergibt sich die dieser Arbeit zugrundeliegende Hauptforschungsfrage:
Welche Architekturmechanismen und Entwurfsentscheidungen sind notwendig, um in einem Java/Spring-basierten Job-Processing-Backend definierte Konsistenzzusicherungen unter typischen Teilausfallszenarien nachweisbar einzuhalten?

Zur Beantwortung dieser Frage werden folgende Teilfragen untersucht:

- TF1: Welche Zusicherungen muss das System gewährleisten, damit Jobs stets korrekt und ohne Inkonsistenzen verarbeitet werden?
- TF2: Welchen Beitrag leisten Idempotenz, Retry-Strategien und Dead-letter-Handling zur Robustheit unter definierten Fehlerszenarien?
- TF3: Wie müssen Transaktionen und Locking-Strategien genutzt werden, damit bei paralleler Arbeit mehrerer Worker keine Doppelverarbeitung oder inkonsistenten Zustände entstehen?
- TF4: Wie lässt sich sicherstellen, dass Jobs nach einem unerwarteten Prozessabbruch korrekt wiederaufgenommen werden, ohne dass Aufträge verloren gehen?

1.3 Zielsetzung

Ziel dieser Arbeit ist der Entwurf, die prototypische Implementierung und die Evaluation eines fehlertoleranten Job-Processing-Backends auf Basis von Java und Spring. Dabei soll herausgearbeitet werden, welche Architekturmechanismen erforderlich sind, damit Hintergrundjobs auch bei Teilausfällen zuverlässig, korrekt und nachvollziehbar verarbeitet werden und definierte Konsistenzzusicherungen eingehalten werden.

Konkret verfolgt die Arbeit folgende Teilziele:

- TZ1: Definition von Zusicherungen: Es sollen präzise Korrektheitsbedingungen formuliert werden, die das System zu jedem Zeitpunkt einhalten muss, unabhängig davon, welche Fehler auftreten. Dazu zählen unter anderem, dass kein Job doppelt erfolgreich abgeschlossen wird, dass kein Job dauerhaft verloren geht und dass Statusübergänge ausschließlich dem definierten Zustandsmodell folgen.

- TZ2: Architekturentwurf: Es soll eine Architektur entworfen werden, die persistente Job-States, Idempotenz, Retry-Strategien sowie ein nachvollziehbares Statusmodell umfasst. Architekturentscheidungen sollen explizit dokumentiert und begründet werden.
- TZ3: Prototypische Implementierung: Die entworfene Architektur soll als funktionsfähiger Prototyp umgesetzt werden, der die definierten Zusicherungen unter realistischen Bedingungen einhält.
- TZ4: Evaluation unter Fehlerbedingungen: Es soll nachgewiesen werden, dass das System seine Zusicherungen auch bei gezielt herbeigeführten Teilausfällen nicht verletzt.

1.4 Hypothesen

Auf Basis der Forschungsfrage und der Zielsetzung werden folgende Hypothesen formuliert, deren Überprüfung im Rahmen der Implementierung und insbesondere der Evaluation erfolgt.

- H1: Durch die Kombination eines persistenten Statusmodells mit transaktionsbasiertem Locking und Idempotenz-Mechanismen lässt sich ein System entwerfen, das auch bei unerwarteten Prozessabbrüchen keine Jobs verliert und keine doppelten Ergebnisse erzeugt.
- H2: Retry-Strategien mit exponentiellem Backoff in Verbindung mit einer Fehlerklassifikation (transient vs. permanent) erhöhen die Robustheit des Systems gegenüber kurzzeitigen Infrastrukturausfällen, ohne dabei die definierten Zusicherungen zu verletzen.
- H3: Die definierten Zusicherungen lassen sich durch automatisierte Tests mit gezielter Fehlerinjektion reproduzierbar überprüfen und nachweisen.

1.5 Aufbau der Arbeit

2 Grundlagen

Orientierung ca. 8-12 Seiten

2.1 Hintergrundverarbeitung in verteilten Systemen

2.1.1 Synchrone vs. Asynchrone Verarbeitung

2.1.2 Background Job-Processing

2.2 Konsistenz und konkurrierende Verarbeitung

2.2.1 ACID

2.2.2 Locking

2.2.3 Transaktionsgrenzen

2.3 Fehlertoleranz und Wiederherstellung

2.3.1 Retry-/Backoff-Strategien

2.3.2 Fehlerklassifikationen

2.3.3 Recovery

2.3.4 Idempotenz

2.4 Zustandsmodellierung mittels State Machines

2.5 Stand der Technik

3 Methodik

3.1 Literaturrecherche

3.2 Ergebnisgenerierung

4 Anforderungen

4.1 Funktionale Anforderungen

4.2 Qualitätsanforderungen

4.3 Zusicherungen

5 Umsetzung

5.1 Architektur und Design

5.2 Implementierung

5.2.1 Technologie-Stack

5.2.2 Zentrale Code-Entscheidungen

6 Evaluation

6.1 Fehlerszenarien

6.2 Durchführung

6.3 Ergebnisse

7 Diskussion

7.1 Gewonnene Erkenntnisse

7.2 Einschränkungen und Grenzen der Arbeit

8 Fazit und Ausblick

Literaturverzeichnis

- Broadcom Inc. (Hrsg.) (24. Mai 2026). *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (besucht am 24.05.2026).
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O'Reilly.
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Terracotta Inc. (Hrsg.) (24. Mai 2026). *Quartz Scheduler Documentation*. URL: <https://www.quartz-scheduler.org/documentation/quartz-2.5.x/> (besucht am 24.05.2026).